

PDF Vectorisation Pipeline

Language	Python or Node.js
Vector DB	Your choice

OVERVIEW

Build a pipeline with two components: (1) an ingestion script that extracts text from a PDF, chunks it, generates vector embeddings, and stores them in a vector database; and (2) a retrieval endpoint that accepts a plain-text query and returns the most semantically similar chunks from the stored embeddings. There is no LLM call on the retrieval side and no conversational layer. Return raw chunk results only.

PART 1 - INGESTION

Write a script that accepts a PDF file path and executes the following steps in order:

1. Extract all text from the PDF, preserving page numbers. Handle multi-page documents.
2. Split the extracted text into overlapping chunks. Chunk size and overlap are your decision - justify both in the README.
3. Generate a vector embedding for each chunk using any embedding model (e.g. OpenAI text-embedding-3-small, sentence-transformers, Cohere Embed).
4. Store each embedding together with its chunk text and source page number in a vector database of your choice (e.g. ChromaDB, FAISS, Pinecone, pgvector).

The script must be runnable as:

```
python ingest.py --file document.pdf
```

PART 2 - RETRIEVAL ENDPOINT

Expose a single HTTP endpoint that embeds the incoming query using the same model used during ingestion and performs a similarity search against the stored vectors.

Method	Endpoint	Description
POST	/query	Request body: { "query": string, "top_k": int }. Response: array of { chunk_text, page_number, score }.

Retrieval must use vector similarity search against the database. Do not use keyword search, SQL LIKE, or full-text search of any kind.

CONSTRAINTS

- No LangChain, LlamaIndex, or any agent/RAG framework. Use raw SDK and library calls only.
- Any PDF parsing library is permitted (e.g. PyMuPDF, pdfplumber, pdf-parse).
- Any embedding model and any vector database are permitted. State and justify your choices in the README.
- The pipeline must run successfully on the sample PDF provided on the assessment platform.

DELIVERABLES

- Ingestion script and retrieval server, fully runnable from your README instructions.
- README: setup steps, justification for chunk size and overlap, justification for embedding model and vector DB, and sample output from the provided PDF.
- results.json: output of at least three different queries run against the provided PDF, in the response format above.

EVALUATION

Criterion	Weight	What we look for
Pipeline runs end-to-end without errors	35%	Ingestion completes; retrieval returns relevant results.
Chunking and embedding design decisions	25%	Choices are justified with reasoning in the README.
Retrieval quality - relevance of returned chunks	25%	results.json entries are semantically relevant to each query.
Code clarity and README completeness	15%	Readable code; setup instructions work on a clean machine.

Instructions

Kindly create a private GitHub repository named: {yourName_Nestack_Submission} (e.g., roy_Nestack_Submission) and add the following contributors for evaluation:

- bishal@nestack.com
- sannidhya@nestack.com
- sanjay@nestack.com

Your submission must include:

- A ZIP file of your complete codebase
- A live deployment link
- Any necessary credentials if your project includes authentication

Additionally, ensure your repository includes a README.md with:

- The deployment link clearly mentioned
- Setup and run instructions if applicable

NOTE: PUBLIC REPOSITORIES ARE DISQUALIFIED FROM THE HIRING PROCESS